

# K0 Grammar Reference

Kotlin Subset Parser — Bison/Flex Implementation

CSE423 Compilers • Final working grammar • 0 S/R conflicts, 0 R/R conflicts

## 1. Overview & Design Decisions

K0 is a statically-typed subset of Kotlin parsed by a hand-crafted Bison/Flex grammar with a custom Automatic Semicolon Insertion (ASI) layer. The grammar was engineered to reach **0 shift/reduce and 0 reduce/reduce conflicts** while covering all K0 language features.

| Property          | Value                                          |
|-------------------|------------------------------------------------|
| Parser generator  | Bison (LALR(1))                                |
| Lexer generator   | Flex                                           |
| ASI layer         | asi.c wrapper around flex_yylex()              |
| Conflicts (final) | 0 S/R, 0 R/R                                   |
| Generic types     | Not supported (avoids LANGLE/RANGLE ambiguity) |
| Nullable types    | IDENT QUEST (e.g. Int?)                        |

**Key insight:** if/when are NOT primary expressions. They live in statement and control\_structure\_body only, preventing double-derivation reduce/reduce conflicts.

## 2. Conflict Resolution History

The grammar started with 97 S/R and 58 R/R conflicts. All were resolved systematically:

| # | Conflict                      | Root Cause                                                | Fix Applied                                                           |
|---|-------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|
| 1 | ~30 S/R on expression LHS     | expression: expression = expression is self-referential   | Changed to: expression: disjunction = expression                      |
| 2 | ~20 R/R on if/when everywhere | if_expression reachable via primary_expr AND directly     | Removed from primary_expr; kept in control_structure_body + statement |
| 3 | ~20 S/R in when entries       | when_entry body had optional SEMI                         | Made SEMI required in when_entry (ASI always provides it)             |
| 4 | 20 R/R + useless rule warning | ELSE if_expression duplicated ELSE control_structure_body | Removed explicit ELSE if_expression rule                              |
| 5 | 3 S/R in do-while             | DO optional_body WHILE — epsilon made WHILE ambiguous     | Changed to DO block WHILE   DO SEMI WHILE                             |
| 6 | 1 S/R in value_argument       | Named-arg IDENT = expr duplicated assignment expression   | Removed named-arg alternative; value_argument: expression only        |

## 3. Grammar Rules

---

### 3.1 Top-Level Structure

```
program:
    import_list top_level_object_list

import_list:
    import_list import_declaration | ε

import_declaration:
    IMPORT import_path | IMPORT import_path SEMI

import_path:
    IDENT | import_path DOT IDENT

top_level_object_list:
    top_level_object_list top_level_object | ε

top_level_object:
    declaration | declaration SEMI
```

### 3.2 Declarations

```
declaration:
    function_declaration | property_declaration

function_declaration:
    FUN IDENT function_value_parameters optional_return_type optional_function_body

optional_return_type:
    COLON type | ε

function_value_parameters:
    LPAREN RPAREN
    LPAREN function_value_parameter_list RPAREN

function_value_parameter:
    IDENT COLON type
    IDENT COLON type ASSIGNMENT expression // default value

function_body:
    block | ASSIGNMENT expression // single-expression body

property_declaration:
    VAL IDENT optional_type_annotation optional_initializer
    VAR IDENT optional_type_annotation optional_initializer
    CONST VAL IDENT optional_type_annotation optional_initializer

optional_initializer:
    ASSIGNMENT expression | ε
```

### 3.3 Types

```
type:
    IDENT // e.g. Int, String, List
    IDENT QUEST // nullable, e.g. Int?

// NOTE: Generic types (e.g. List<Int>) are NOT supported.
// They would require disambiguating LANGLE/RANGLE as
// type brackets vs comparison operators.
```

### 3.4 Blocks & Statements

```
block:
    LCURL statements RCURL

statements:
    ε
    statement_list
    statement_list SEMI

statement_list:
    statement
    statement_list SEMI statement

statement:
    declaration
    loop_statement
    if_expression           // direct - NOT via expression tower
    when_expression        // direct - NOT via expression tower
    expression
    return_statement
    break_statement
    continue_statement
```

**Why if/when in statement?** We removed if\_expression from primary\_expr to fix R/R conflicts. Without adding it back to statement, any 'if' at the start of a block would have no valid parse path.

### 3.5 Loop Statements

```
loop_statement:
    for_statement | while_statement | do_while_statement

for_statement:
    FOR LPAREN IDENT IN expression RPAREN optional_control_structure_body
    FOR LPAREN IDENT COLON type IN expression RPAREN optional_control_structure_body

while_statement:
    WHILE LPAREN expression RPAREN control_structure_body
    WHILE LPAREN expression RPAREN SEMI           // empty body

do_while_statement:
    DO block WHILE LPAREN expression RPAREN
    DO SEMI WHILE LPAREN expression RPAREN        // empty body

// do-while NEVER uses optional_control_structure_body.
// Epsilon body caused WHILE to be ambiguous after DO.
// ASI suppresses SEMI between '}' and 'while' (see Section 5).
```

### 3.6 Control Structure Body

```
control_structure_body:
    block
    if_expression
    when_expression
    simple_statement

simple_statement:
    loop_statement | expression
    return_statement | break_statement | continue_statement
```

```
optional_control_structure_body:
    control_structure_body | ε
```

### 3.7 If and When Expressions

```
if_expression:
    IF LPAREN expression RPAREN control_structure_body %prec ELSE
    IF LPAREN expression RPAREN control_structure_body
    ELSE control_structure_body

// ELSE if_expression is handled implicitly:
// ELSE control_structure_body → ELSE if_expression
// An explicit ELSE if_expression rule would duplicate this
// derivation and cause R/R conflicts.

when_expression:
    WHEN LCURL when_entry_list RCURL
    WHEN LPAREN expression RPAREN LCURL when_entry_list RCURL

when_entry_list:
    when_entry_list when_entry | ε

when_entry:
    when_condition_list ARROW control_structure_body SEMI
    ELSE ARROW control_structure_body SEMI

// SEMI is required (not optional) after when_entry body.
// Without it, tokens starting a new when_condition are ambiguous
// with tokens continuing the body expression. ASI always inserts SEMI.

when_condition_list:
    when_condition | when_condition_list COMMA when_condition

when_condition:
    expression
```

### 3.8 Jump Statements

```
return_statement:
    RETURN | RETURN expression

break_statement:    BREAK
continue_statement: CONTINUE
```

## 4. Expression Tower

Expressions are stratified from lowest to highest precedence. Each level delegates to the level above it, with operators inserted at the appropriate tier.

| Lvl | Nonterminal   | Operators        | Notes                                                                  |
|-----|---------------|------------------|------------------------------------------------------------------------|
| 1   | expression    | = += -= *= /= %= | LHS is disjunction (not expression)<br>— prevents self-referential S/R |
| 2   | disjunction   |                  | Left-associative                                                       |
| 3   | conjunction   | &&               | Left-associative                                                       |
| 4   | equality_expr | == != === !==    | Left-associative                                                       |

|    |                     |                             |                             |
|----|---------------------|-----------------------------|-----------------------------|
| 5  | comparison_expr     | < > <= >=                   | Left-associative            |
| 6  | infix_operation     | in                          | Keyword operator            |
| 7  | elvis_expr          | ?:                          | Right-associative           |
| 8  | range_expr          | .. ..<                      | Left-associative            |
| 9  | additive_expr       | + -                         | Left-associative            |
| 10 | multiplicative_expr | * / %                       | Left-associative            |
| 11 | as_expr             | as                          | Type cast, left-assoc       |
| 12 | prefix_expr         | ! - + ++ --                 | Right-associative, unary    |
| 13 | postfix_expr        | ++ -- . ?. :: [] ()         | Highest precedence postfix  |
| 14 | primary_expr        | literals, IDENT, (expr), [] | Base case — no if/when here |

## 4.1 Primary Expressions

```
primary_expr:
    LPAREN expression RPAREN    // grouped
    IDENT
    literal
    STRINGLITERAL
    collection_literal

// if_expression and when_expression are intentionally ABSENT.
// Their presence caused 58 R/R conflicts (two paths to same node).

literal:
    INTEGERLITERAL | LONGLITERAL | REALLITERAL | DOUBLELITERAL
    BOOLEANLITERAL | CHARACTERLITERAL | NULLLITERAL

collection_literal:
    LSQUARE RSQUARE
    LSQUARE collection_items RSQUARE

collection_items:
    expression | collection_items COMMA expression
```

## 4.2 Postfix & Call Expressions

```
postfix_expr:
    primary_expr
    postfix_expr INCR           // x++
    postfix_expr DECR          // x--
    postfix_expr DOT IDENT     // x.field
    postfix_expr SAFE_CALL IDENT // x?.field
    postfix_expr DOUBLE_COLON IDENT // X::method
    postfix_expr LSQUARE expression RSQUARE // x[i]
    postfix_expr LPAREN optional_value_arguments RPAREN // f(args)

value_argument:
    expression

// Named arguments (IDENT = expr) were removed — they duplicate
// assignment expressions and cause 1 S/R conflict.
```

## 5. Automatic Semicolon Insertion (ASI)

ASI is implemented in `asi.c` as a wrapper around the Flex-generated `flex_yylex()`. When a NEWLINE token is encountered, ASI decides whether to emit a SEMI token before the next real token.

## 5.1 Tokens That Trigger SEMI

A SEMI is emitted on newline if the previous token is one of:

```
IDENT
INTEGERLITERAL  LONGLITERAL  REALLITERAL  DOUBLELITERAL
BOOLEANLITERAL  CHARACTERLITERAL  STRINGLITERAL  NULLLITERAL
RETURN  BREAK  CONTINUE
INCR  DECR
RPAREN  RSQUARE
RCURL  // special — see Section 5.2
```

## 5.2 RCURL Special Case (do-while)

**The problem:** RCURL must trigger SEMI so statements after loop/if bodies are separated. But in `do { } while(...)`, emitting SEMI between `}` and `while` would break the do-while rule.

Solution: RCURL is in the trigger list, but SEMI is **suppressed** when the next real token is WHILE:

```
if (last_tok == RCURL && next == WHILE) {
    // do { ... } while(...) — don't insert SEMI
    return WHILE;
}
```

## 5.3 LCURL Suppression

```
if (next == LCURL) {
    // Never insert SEMI before '{'
    // e.g. fun foo() { ... } should not get a SEMI before {
    return LCURL;
}
```

## 5.4 ASI Algorithm Summary

| Previous token                         | Next token    | Action                       |
|----------------------------------------|---------------|------------------------------|
| <code>can_end_statement = true</code>  | WHILE         | Suppress SEMI (do-while)     |
| <code>can_end_statement = true</code>  | LCURL         | Suppress SEMI (block open)   |
| <code>can_end_statement = true</code>  | anything else | Emit SEMI, buffer next token |
| <code>can_end_statement = false</code> | (any)         | Consume newline, continue    |

## 6. Operator Precedence & Associativity

Declared in the `%left / %right / %nonassoc` section of `k0gram.y`, from lowest to highest:

| Pri | Tokens | Assoc | Purpose |
|-----|--------|-------|---------|
|-----|--------|-------|---------|

|              |                         |        |                                              |
|--------------|-------------------------|--------|----------------------------------------------|
| 1<br>(low)   | = += -= *= /= %=        | %right | Assignment (LHS: disjunction)                |
| 2            |                         | %left  | Logical OR                                   |
| 3            | &&                      | %left  | Logical AND                                  |
| 4            | == != === !==           | %left  | Equality                                     |
| 5            | < > <= >=               | %left  | Comparison                                   |
| 6            | in                      | %left  | In-range / collection membership             |
| 7            | ?:                      | %right | Elvis operator                               |
| 8            | .. ..<                  | %left  | Range / range-until                          |
| 9            | + -                     | %left  | Addition / subtraction                       |
| 10           | * / %                   | %left  | Multiplication / division / modulo           |
| 11           | as                      | %left  | Type cast                                    |
| 12           | ! - + ++ --<br>(prefix) | %right | Unary prefix<br>(UMINUS/UPLUS/PREINC/PREDEC) |
| 13<br>(high) | ++ -- . ?. :: [] ()     | %left  | Postfix / member access / call               |
| —            | else                    | %right | Dangling-else resolution                     |

## 7. Token Reference

### 7.1 Keywords

```
FUN VAL VAR IF ELSE WHILE FOR RETURN BREAK CONTINUE
WHEN IN DO IMPORT CONST AS
```

### 7.2 Literals

```
INTEGERLITERAL  LONGLITERAL    REALLITERAL    DOUBLELITERAL
BOOLEANLITERAL  CHARACTERLITERAL STRINGLITERAL NULLLITERAL
```

### 7.3 Operators & Punctuation

| Token      | Text | Token  | Text |
|------------|------|--------|------|
| ASSIGNMENT | =    | ADD    | +    |
| ADD_ASSIGN | +=   | SUB    | -    |
| SUB_ASSIGN | -=   | MUL    | *    |
| MUL_ASSIGN | *=   | DIV    | /    |
| DIV_ASSIGN | /=   | MOD    | %    |
| MOD_ASSIGN | %=   | INCR   | ++   |
| EQ         | ==   | DECR   | --   |
| NEQ        | !=   | NOT    | !    |
| GTE        | >=   | LANGLE | <    |
| LTE        | <=   | RANGLE | >    |

|           |     |              |           |
|-----------|-----|--------------|-----------|
| REF_EQ    | === | REF_NEQ      | !==       |
| AND       | &&  | OR           |           |
| SAFE_CALL | ?.  | ELVIS        | ?:        |
| RANGE     | ..  | RANGE_UNTIL  | ..<br>..< |
| LPAREN    | (   | RPAREN       | )         |
| LCURL     | {   | RCURL        | }         |
| LSQUARE   | [   | RSQUARE      | ]         |
| COMMA     | ,   | DOT          | .         |
| COLON     | :   | SEMI         | ;         |
| ARROW     | ->  | DOUBLE_COLON | ::        |
| QUEST     | ?   | NEWLINE      | \n        |

## 8. Known Limitations

| Feature                         | Status / Workaround                                      |
|---------------------------------|----------------------------------------------------------|
| Generic types (List<Int>)       | Not supported — LANGLE/RANGLE ambiguity with comparisons |
| Named arguments foo(x = 1)      | Not supported — duplicates assignment expression         |
| Lambda expressions { x -> ... } | Not supported                                            |
| String templates "Hello \$name" | Treated as opaque STRINGLITERAL                          |
| Companion objects / classes     | Not supported                                            |
| Extension functions             | Not supported                                            |
| Multiline strings """..."""     | Not supported                                            |

## 9. Test Coverage

The grammar was validated against `Test_full.kt`, a comprehensive single-file test covering all supported features:

| Feature                  | Test construct                                       |
|--------------------------|------------------------------------------------------|
| Import declarations      | import kotlin.math, import java.util.ArrayList       |
| Basic functions          | fun main(), fun add(x: Int, y: Int): Int             |
| All literal types        | Int, Long, Real, Double, Boolean, Char, String, null |
| All assignment operators | = += -= *= /= %=                                     |
| Prefix/postfix operators | ++ -- (both positions), unary - +                    |
| If / else-if / else      | Full chain, nested                                   |
| While loop               | Condition + block body                               |
| Do-while loop            | Block body, WHILE on same line as }                  |
| For-in loop              | Plain and with type annotation                       |



|                            |                                  |
|----------------------------|----------------------------------|
| When without subject       | Multiple conditions, else branch |
| When with subject          | Single expression subject        |
| Dot chaining               | a.b.c, safe call ?.              |
| Indexing                   | a[i]                             |
| Collection literals        | [1, 2, 3]                        |
| Elvis operator             | a ?: b                           |
| Range operator             | 1..10, x in 1..10                |
| Type cast                  | x as Int                         |
| Return / break / continue  | All positions                    |
| Single-expression function | fun double(x: Int) = x * 2       |
| Nullable types             | Int?, String?                    |
| const val                  | Top-level constant               |